

Centennial College

Group Project – Enterprise eCommerce Database Design

Introduction To Database Conce

Shiori Takeda - 301545004

You-chen Cheng - 301540029

Yin Hua Li

08.07.2026



Phase 1 :

1-1. Stake Holder

Name:	Can:
Customer	- buy a product - check a product - search a product - ask customer service
System Administrator	- manage eCommerce - modify bug - keep security
Database Administrator	- add a new product - delete an old product - reduce a duplicate
Shop Manager	- manage shop's product - manage online product - exchange information each shop - send or receive a stock - manage ship date
Shop Employee	- check shop's stock - record product information which is sold - store new product's information
Vendor	- manage a stock in the company - supply a product to eCommerce - receive return

1-2. Functional and Non-functional Requirements

Functional Requirements:	Role:
Shopping Cart	- store customers' selection - lock product data for a while - delete data regarding customers' decision
Search	- access product database related to the keyword - analyze typo and predict a location
Product's Detail	- output products' information (e.g., name, vendor and price)
Result	- output relevant product data
Category	- list category data
Order	- show products' data in shopping cart
Checkout	- create new order_id and store in database
Cancel	- delete order items
Return	- delete order items and store in the list of return items
Home	- pick category and random order items from database
Register	- store customers' information
Track shipments	- pick orderdate and shipdate up, calculate them and show virtual columns
Wish lists	- store customers' favorite items
Employees generate monthly reports	- store employees' reports per month - record the change in each entity

Non-functional Requirements:	Role:
Access Speed [Fast product searching]	- support fast retrieval of product information - allow a user to search and view products with minimal response time
Data integrity	- maintain accurate and consistent data between the website and database
Minimize redundancy using normalization	- be designed using normalization techniques to reduce duplicate data - improve storage efficiency - prevent update anomalies
Secure customer information	- protect customer data through authentication, authorization, encryption where appropriate, and access controls - be accessible to an authorized user
Oracle SQL compatible	- be compatible with Oracle SQL - ensure the system can be deployed and maintained in an Oracle Database environment

1-3. Study Existing Systems

References:	Link:
Amazon.com	https://www.amazon.ca/
Rakuten (Japanese eCommerce)	https://www.rakuten.co.jp/
BEST BUY	https://www.bestbuy.ca/en-ca
Walmart (Online)	https://www.walmart.com/

1-4. Use Case and Workflows

UC-01: Customer Registration & Account Management

- Primary Actor: Customer
- Goal: Create and update a user account in the system.
- Pre-conditions: None.
- Post-conditions: A new customer record is stored in the database.
- Target Database Tables: ECUSTOMERS
- Main Success Scenario:
 - Customer enters personal details (First Name, Last Name, Email, Address).
 - System verifies that the provided email is unique.
 - System assigns a unique customer_id and registers registerdate using SYSDATE.

UC-02: Order Placement & Inventory Allocation

- Primary Actor: Customer
- Goal: Purchase items, create an order record, and adjust store inventory.
- Pre-conditions: Customer is registered; selected items have sufficient quantity in the ESTOCK table.
- Post-conditions: Records inserted into EORDERS and EORDERITEMS; ESTOCK updated.
- Target Database Tables: ECUSTOMERS, EITEMS, EORDERS, EORDERITEMS, ESTOCK
- Main Success Scenario:
 - Customer selects one or more items (item_id) and quantities.
 - System verifies available quantity in ESTOCK for the designated store_id.
 - Customer provides shipping address (shipaddress) and confirms payment.
 - System generates a new order_id in EORDERS.
 - System inserts itemized rows into EORDERITEMS with historical unit price (payeach).
 - System updates ESTOCK table by deducting the purchased quantities (quantity = quantity - Item_quantity).

UC-03: Product Return & Vendor Stock Logging

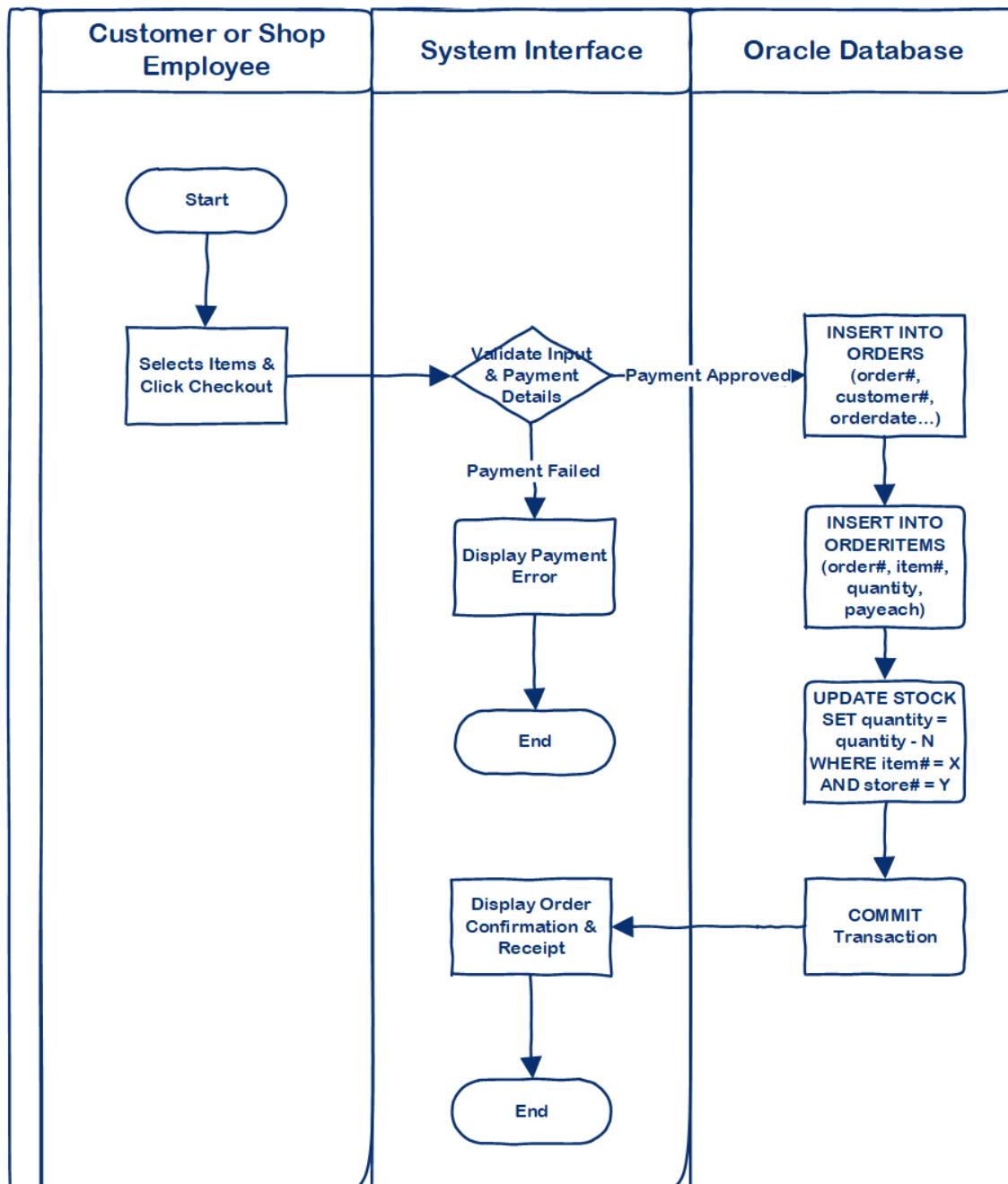
- Primary Actor: Shop Employee / Customer Service
- Goal: Process a returned item and log return metrics for the supplying company.

- **Pre-conditions:** The item was previously purchased in a valid ORDERS record.
- **Post-conditions:** A return record is created in RETURNSTOCK.
- **Target Database Tables:** ERETURNSTOCK, EITEMS, ECOMPANY, ESTORE
- **Main Success Scenario:**
 - Employee receives the returned item (*item_id*) from the customer at a specific *store_id*.
 - System retrieves the vendor details (*company_id*) linked to the item from EITEMS.
 - System inserts a record into ERETURNSTOCK (*return_id*, *item_id*, *company_id*, *store_id*, *returndate*).
 - System updates store stock or flags item for vendor write-off.

UC-04: Category Promotion & Gift Assignment

- **Primary Actor:** Shop Manager / Marketing Admin
- **Goal:** Assign automated gifts and price-range promotions to product categories.
- **Pre-conditions:** Valid gifts exist in the EGIFT table.
- **Post-conditions:** Promotion and category linkages are established.
- **Target Database Tables:** EGIFT, EPROMOTION, ECATEGORY, EITEMS
- **Main Success Scenario:**
 - Manager sets up a promotional threshold (*minretail*, *maxretail*) in EPROMOTION and links a *gift_id*.
 - Manager assigns the *promotion_id* to a specific *category_id* in ECATEGORY.
 - System automatically applies eligibility rules for items belonging to that category.

Order Fulfillment & Inventory Deduction

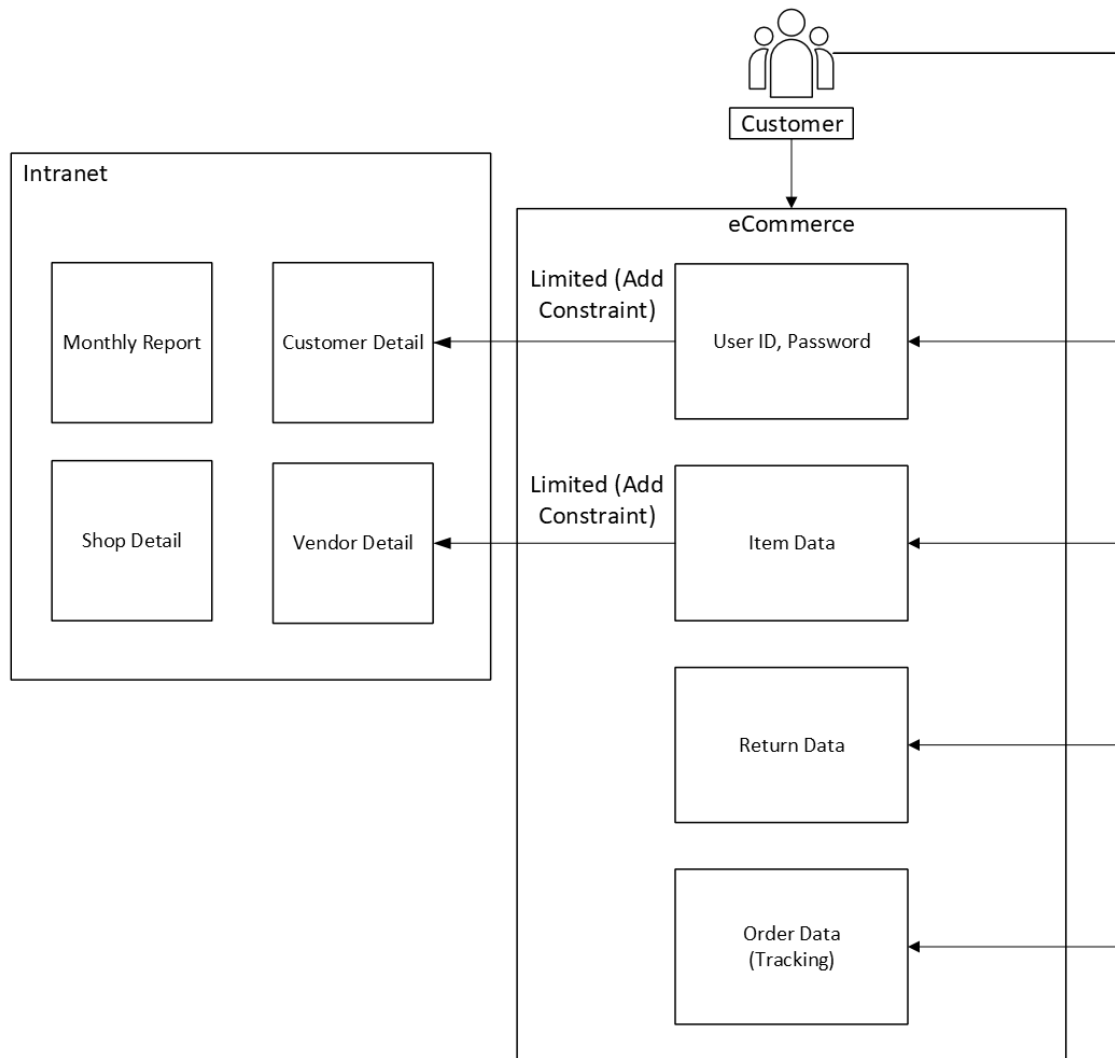


Phase 2 :

2-1, 2. Stakeholder Class and Data Access Rights or Specific System Boundaries

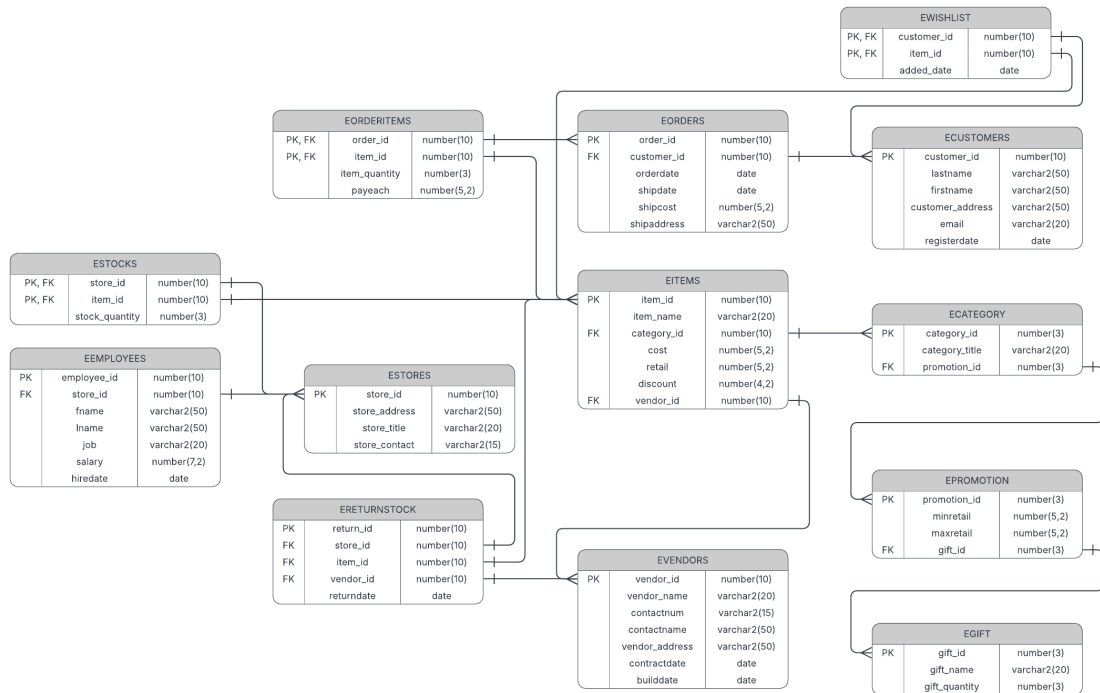
Name:	Can:
Customer	- buy a product - check a product - search a product - ask customer service
System Administrator	- manage eCommerce - modify bug - keep security
Database Administrator	- add a new product - delete an old product - reduce a duplicate
Shop Manager	- manage shop's product - manage online product - exchange information each shop - send or receive a stock - manage ship date
Shop Employee	- check shop's stock - record product information which is sold - store new product's information
Vendor	- manage a stock in the company - supply a product to eCommerce - receive return

2-3. Map out custom user interfaces, view windows, and data sub-schemas required for different actors (e.g., product search for customers vs. report generation for managers).



Phase 3 :

3-1,2,3. Entities, Attributes, Relationships



Phase 4 :

Tables:

```
CREATE TABLE eCustomers
(customer_id NUMBER(10) PRIMARY KEY,
lastname VARCHAR2(50) NOT NULL,
firstname VARCHAR2(50) NOT NULL,
customer_address VARCHAR2(50),
email VARCHAR2(20) NOT NULL,
registerdate DATE NOT NULL);
```

```
CREATE TABLE eOrders
(order_id NUMBER(10) PRIMARY KEY,
customer_id NUMBER(10) NOT NULL,
orderdate DATE NOT NULL,
shipdate DATE NOT NULL,
shipcost NUMBER(5,2) NOT NULL,
shipaddress VARCHAR2(50) NOT NULL,
CONSTRAINT eorders_customerid_fk FOREIGN KEY (customer_id)
REFERENCES eCustomers (customer_id),
CONSTRAINT eorders_shipdate_ck CHECK (shipdate>=orderdate));
```

```
CREATE TABLE eStores
(store_id NUMBER(10) PRIMARY KEY,
store_address VARCHAR2(50) NOT NULL,
store_title VARCHAR2(20) NOT NULL,
store_contact VARCHAR2(15) NOT NULL);
```

```
CREATE TABLE eEmployees
(employee_id NUMBER(10) PRIMARY KEY,
store_id NUMBER(10),
fname VARCHAR2(50) NOT NULL,
lname VARCHAR2(50) NOT NULL,
job VARCHAR2(20) NOT NULL,
manager_id NUMBER(10),
salary NUMBER(7,2) NOT NULL,
```

```
hiredate DATE NOT NULL,
    CONSTRAINT eemployees_storeid_fk FOREIGN KEY (store_id) REFERENCES
eStores (store_id));
```

```
CREATE TABLE eVendors
(vendor_id NUMBER(10) PRIMARY KEY,
vendor_name VARCHAR2(20) NOT NULL,
contactnumber VARCHAR2(15) NOT NULL,
contactname VARCHAR2(50) NOT NULL,
vendor_address VARCHAR2(50),
contractdate DATE,
builddate DATE,
    CONSTRAINT evendors_builddate_ck CHECK (builddate<=contractdate));
```

```
CREATE TABLE eGift
(gift_id NUMBER(3) PRIMARY KEY,
gift_name VARCHAR2(20) NOT NULL,
gift_quantity NUMBER(3) DEFAULT 0,
    CONSTRAINT egift_quantity_ck CHECK (gift_quantity>=0));
```

```
CREATE TABLE ePromotion
(promotion_id NUMBER(3) PRIMARY KEY,
minretail NUMBER(5,2) NOT NULL,
maxretail NUMBER(5,2) NOT NULL,
gift_id NUMBER(3) NOT NULL,
    CONSTRAINT epromotion_giftid_fk FOREIGN KEY (gift_id) REFERENCES
eGift (gift_id),
    CONSTRAINT epromotion_retail_ck CHECK (minretail<=maxretail));
```

```
CREATE TABLE ecategory
(category_id NUMBER(3) PRIMARY KEY,
category_title VARCHAR2(20) NOT NULL,
promotion_id NUMBER(3),
    CONSTRAINT ecategory_promotionid_fk FOREIGN KEY (promotion_id)
REFERENCES ePromotion (promotion_id));
```

```

CREATE TABLE eItems
(item_id NUMBER(10) PRIMARY KEY,
item_name VARCHAR2(20) NOT NULL,
category_id NUMBER(10),
cost NUMBER(5,2) NOT NULL,
retail NUMBER(5,2) NOT NULL,
discount NUMBER(4,2) DEFAULT 0,
vendor_id NUMBER(10) NOT NULL,
    CONSTRAINT eitems_categoryid_fk FOREIGN KEY (category_id)
REFERENCES eCategory (category_id),
    CONSTRAINT eitems_vendorid_fk FOREIGN KEY (vendor_id) REFERENCES
eVendors (vendor_id),
    CONSTRAINT eitems_discount_ck CHECK (discount>=0),
    CONSTRAINT eitems_cost_ck CHECK (cost<=retail));

```

```

CREATE TABLE eStocks
(store_id NUMBER(10),
item_id NUMBER(10),
stock_quantity NUMBER(3) DEFAULT 0,
    CONSTRAINT estocks_storeid_pk PRIMARY KEY (store_id, item_id),
    CONSTRAINT estocks_storeid_fk FOREIGN KEY (store_id) REFERENCES
eStores (store_id),
    CONSTRAINT estocks_itemid_fk FOREIGN KEY (item_id) REFERENCES
eItems (item_id),
    CONSTRAINT estocks_quantity_ck CHECK (stock_quantity>=0));

```

```

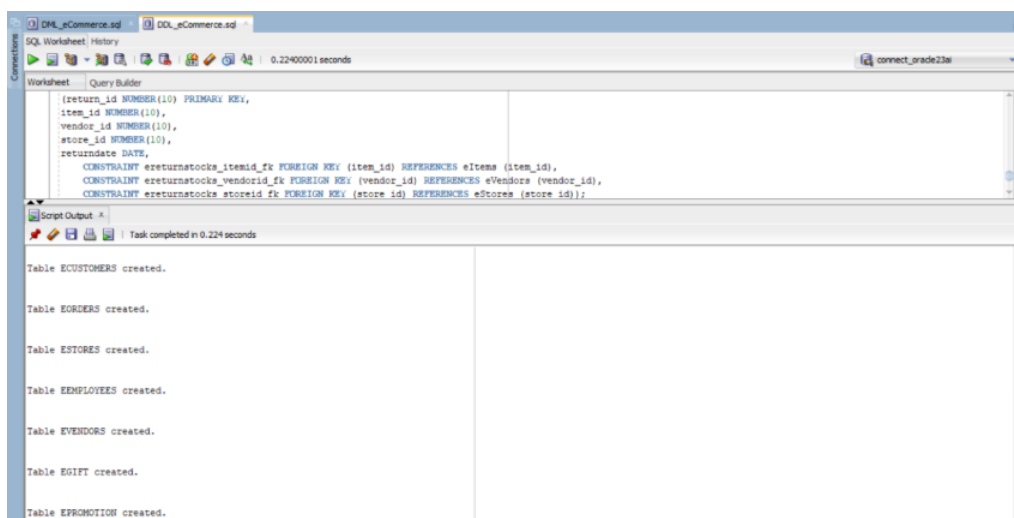
CREATE TABLE eReturnstocks
(return_id NUMBER(10) PRIMARY KEY,
item_id NUMBER(10),
vendor_id NUMBER(10),
store_id NUMBER(10),
returndate DATE,
    CONSTRAINT ereturnstocks_itemid_fk FOREIGN KEY (item_id) REFERENCES
eItems (item_id),
    CONSTRAINT ereturnstocks_vendorid_fk FOREIGN KEY (vendor_id)
REFERENCES eVendors (vendor_id),

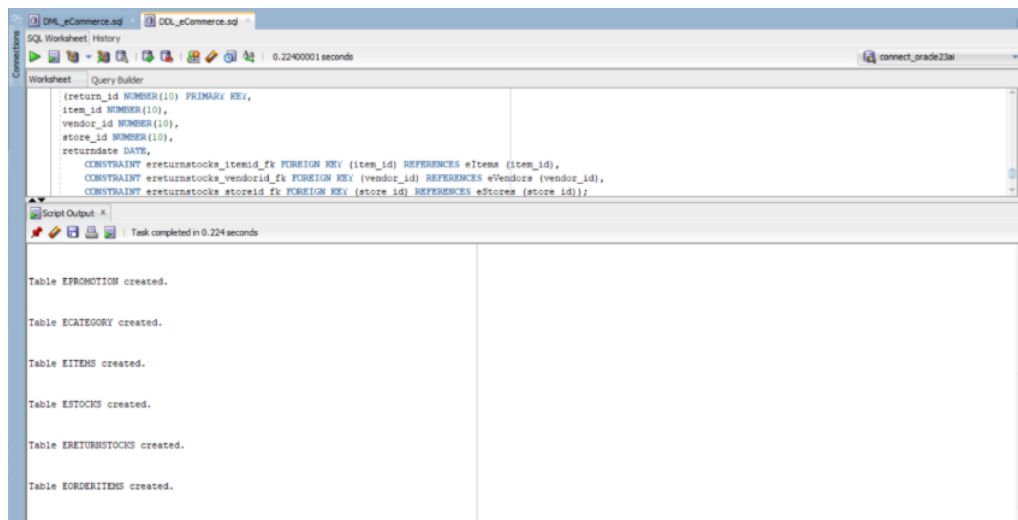
```

```
CONSTRAINT ereturnstocks_storeid_fk FOREIGN KEY (store_id)
REFERENCES eStores (store_id));
```

```
CREATE TABLE eOrderitems
(order_id NUMBER(10),
item_id NUMBER(10),
item_quantity NUMBER(3) NOT NULL,
payeach NUMBER(5,2) NOT NULL,
CONSTRAINT eorderitems_orderid_pk PRIMARY KEY (order_id, item_id),
CONSTRAINT eorderitems_orderid_fk FOREIGN KEY (order_id) REFERENCES
eorders (order_id),
CONSTRAINT eorderitems_itemid_fk FOREIGN KEY (item_id) REFERENCES
eitems (item_id));
```

```
CREATE TABLE ewishlist
(customer_id NUMBER(10),
item_id NUMBER(10),
added_date DATE,
CONSTRAINT ewishlist_customerid_PK PRIMARY KEY
(customer_id, item_id),
CONSTRAINT ewishlist_customerid_FK FOREIGN KEY (customer_id)
REFERENCES ecustomers (customer_id),
CONSTRAINT ewishlist_itemid_FK FOREIGN KEY (item_id)
REFERENCES eitems (item_id));
```





Data:

```

INSERT INTO eCustomers(customer_id, lastname, firstname,
customer_address, email, registerdate) VALUES (1001, 'Smith', 'John',
'123 Maple St', 'john@email.com', DATE '2023-01-15');
INSERT INTO eCustomers(customer_id, lastname, firstname,
customer_address, email, registerdate) VALUES (1002, 'Johnson',
'Emily', '456 Oak Ave', 'emily@email.com', DATE '2023-02-10');
INSERT INTO eCustomers(customer_id, lastname, firstname,
customer_address, email, registerdate) VALUES (1003, 'Williams',
'Michael', '789 Pine Rd', 'michael@email.com', DATE '2023-03-05');
INSERT INTO eCustomers(customer_id, lastname, firstname,
customer_address, email, registerdate) VALUES (1004, 'Brown', 'Sarah',
'321 Cedar Dr', 'sarah@email.com', DATE '2023-03-22');
INSERT INTO eCustomers(customer_id, lastname, firstname,
customer_address, email, registerdate) VALUES (1005, 'Jones', 'David',
'654 Birch Ln', 'david@email.com', DATE '2023-04-18');
INSERT INTO eCustomers(customer_id, lastname, firstname,
customer_address, email, registerdate) VALUES (1006, 'Garcia',
'Sophia', '987 Willow St', 'sophia@email.com', DATE '2023-05-11');
INSERT INTO eCustomers(customer_id, lastname, firstname,
customer_address, email, registerdate) VALUES (1007, 'Miller',
'James', '741 Elm St', 'james@email.com', DATE '2023-06-03');

```

```
INSERT INTO eCustomers(customer_id, lastname, firstname,
customer_address, email, registerdate) VALUES (1008, 'Davis',
'Olivia', '852 Cherry Ave', 'olivia@email.com', DATE '2023-07-09');
INSERT INTO eCustomers(customer_id, lastname, firstname,
customer_address, email, registerdate) VALUES (1009, 'Wilson',
'Daniel', '963 Aspen Ct', 'daniel@email.com', DATE '2023-08-14');
INSERT INTO eCustomers(customer_id, lastname, firstname,
customer_address, email, registerdate) VALUES (1010, 'Taylor', 'Emma',
'159 Spruce Blvd', 'emma@email.com', DATE '1111-11-11');
```

```
INSERT INTO eOrders (order_id, customer_id, orderdate, shipdate,
shipcost, shipaddress) VALUES (2001, 1001, DATE '2023-10-01', DATE
'2023-10-03', 8.99, '123 Maple St');
INSERT INTO eOrders (order_id, customer_id, orderdate, shipdate,
shipcost, shipaddress) VALUES (2002, 1002, DATE '2023-10-02', DATE
'2023-10-05', 12.50, '456 Oak Ave');
INSERT INTO eOrders (order_id, customer_id, orderdate, shipdate,
shipcost, shipaddress) VALUES (2003, 1003, DATE '2023-10-04', DATE
'2023-10-06', 6.75, '789 Pine Rd');
INSERT INTO eOrders (order_id, customer_id, orderdate, shipdate,
shipcost, shipaddress) VALUES (2004, 1004, DATE '2023-10-05', DATE
'2023-10-08', 9.99, '321 Cedar Dr');
INSERT INTO eOrders (order_id, customer_id, orderdate, shipdate,
shipcost, shipaddress) VALUES (2005, 1005, DATE '2023-10-06', DATE
'2023-10-07', 5.50, '654 Birch Ln');
INSERT INTO eOrders (order_id, customer_id, orderdate, shipdate,
shipcost, shipaddress) VALUES (2006, 1006, DATE '2023-10-07', DATE
'2023-10-10', 11.25, '987 Willow St');
INSERT INTO eOrders (order_id, customer_id, orderdate, shipdate,
shipcost, shipaddress) VALUES (2007, 1007, DATE '2023-10-08', DATE
'2023-10-09', 7.95, '741 Elm St');
INSERT INTO eOrders (order_id, customer_id, orderdate, shipdate,
shipcost, shipaddress) VALUES (2008, 1008, DATE '2023-10-09', DATE
'2023-10-12', 13.40, '852 Cherry Ave');
```

```
INSERT INTO eOrders (order_id, customer_id, orderdate, shipdate,
shipcost, shipaddress) VALUES (2009, 1009, DATE '2023-10-10', DATE
'2023-10-13', 10.00, '963 Aspen Ct');
INSERT INTO eOrders (order_id, customer_id, orderdate, shipdate,
shipcost, shipaddress) VALUES (2010, 1010, DATE '2023-10-11', DATE
'2023-10-14', 8.25, '159 Spruce Blvd');
```

```
INSERT INTO eVendors (vendor_id, vendor_name, contactnumber,
contactname, vendor_address, contractdate, builddate) VALUES (101,
'TechSupply', '555-1001', 'Alice Brown', '12 Market St', DATE '2023-
06-15', DATE '2018-04-10');
INSERT INTO eVendors (vendor_id, vendor_name, contactnumber,
contactname, vendor_address, contractdate, builddate) VALUES (102,
'OfficePlus', '555-1002', 'Brian Smith', '45 King Ave', DATE '2023-07-
10', DATE '2019-02-18');
INSERT INTO eVendors (vendor_id, vendor_name, contactnumber,
contactname, vendor_address, contractdate, builddate) VALUES (103,
'GadgetWorld', '555-1003', 'Carol Davis', '78 Queen Rd', DATE '2023-
08-05', DATE '2020-05-12');
INSERT INTO eVendors (vendor_id, vendor_name, contactnumber,
contactname, vendor_address, contractdate, builddate) VALUES (104,
'HomeGoods', '555-1004', 'David Wilson', '91 Elm St', DATE '2023-06-
20', DATE '2017-09-30');
INSERT INTO eVendors (vendor_id, vendor_name, contactnumber,
contactname, vendor_address, contractdate, builddate) VALUES (105,
'FurnitureCo', '555-1005', 'Emma Taylor', '23 Oak Dr', DATE '2023-09-
12', DATE '2016-11-22');
INSERT INTO eVendors (vendor_id, vendor_name, contactnumber,
contactname, vendor_address, contractdate, builddate) VALUES (106,
'PrintMaster', '555-1006', 'Frank Miller', '56 Pine Blvd', DATE '2023-
07-18', DATE '2019-08-15');
INSERT INTO eVendors (vendor_id, vendor_name, contactnumber,
contactname, vendor_address, contractdate, builddate) VALUES (107,
'DigitalHub', '555-1007', 'Grace Johnson', '89 Cedar Ave', DATE '2023-
10-01', DATE '2021-01-05');
```

```

INSERT INTO eVendors (vendor_id, vendor_name, contactnumber,
contactname, vendor_address, contractdate, builddate) VALUES (108,
'AccessoryPro', '555-1008', 'Henry Moore', '14 Birch Ln', DATE '2023-
08-28', DATE '2020-06-10');
INSERT INTO eVendors (vendor_id, vendor_name, contactnumber,
contactname, vendor_address, contractdate, builddate) VALUES (109,
'SmartDevices', '555-1009', 'Isabella White', '37 Willow Ct', DATE
'2023-09-30', DATE '2018-12-08');
INSERT INTO eVendors (vendor_id, vendor_name, contactnumber,
contactname, vendor_address, contractdate, builddate) VALUES (110,
'ComputerZone', '555-1010', 'Jack Harris', '62 Maple Rd', DATE '2023-
10-15', DATE '2017-03-25');

```

```

INSERT INTO eGift (gift_id, gift_name, gift_quantity) VALUES (1, 'Free
Mouse', 50);
INSERT INTO eGift (gift_id, gift_name, gift_quantity) VALUES (2, 'USB
Cable', 100);
INSERT INTO eGift (gift_id, gift_name, gift_quantity) VALUES (3,
'Notebook', 75);
INSERT INTO eGift (gift_id, gift_name, gift_quantity) VALUES (4, 'Pen
Set', 120);
INSERT INTO eGift (gift_id, gift_name, gift_quantity) VALUES (5,
'Phone Stand', 60);
INSERT INTO eGift (gift_id, gift_name, gift_quantity) VALUES (6,
'Screen Cleaner', 80);
INSERT INTO eGift (gift_id, gift_name, gift_quantity) VALUES (7,
'Laptop Bag', 40);
INSERT INTO eGift (gift_id, gift_name, gift_quantity) VALUES (8,
'Keyboard Cover', 90);
INSERT INTO eGift (gift_id, gift_name, gift_quantity) VALUES (9,
'Coffee Mug', 55);
INSERT INTO eGift (gift_id, gift_name, gift_quantity) VALUES (10,
'Sticker Pack', 150);

```

```

INSERT INTO eStores (store_id, store_address, store_title,
store_contact) VALUES (4001, '123 Main St', 'Downtown', '555-1001');

```

```

INSERT INTO eStores (store_id, store_address, store_title,
store_contact) VALUES (4002, '45 King Rd', 'Westside', '555-1002');
INSERT INTO eStores (store_id, store_address, store_title,
store_contact) VALUES (4003, '89 Queen Ave', 'Eastside', '555-1003');
INSERT INTO eStores (store_id, store_address, store_title,
store_contact) VALUES (4004, '12 River St', 'North Mall', '555-1004');
INSERT INTO eStores (store_id, store_address, store_title,
store_contact) VALUES (4005, '77 Lake Dr', 'South Plaza', '555-1005');
INSERT INTO eStores (store_id, store_address, store_title,
store_contact) VALUES (4006, '91 Hill Rd', 'Central', '555-1006');
INSERT INTO eStores (store_id, store_address, store_title,
store_contact) VALUES (4007, '63 Park Ave', 'Riverside', '555-1007');
INSERT INTO eStores (store_id, store_address, store_title,
store_contact) VALUES (4008, '28 Green St', 'City Mall', '555-1008');
INSERT INTO eStores (store_id, store_address, store_title,
store_contact) VALUES (4009, '14 Oak Blvd', 'Uptown', '555-1009');
INSERT INTO eStores (store_id, store_address, store_title,
store_contact) VALUES (4010, '55 Sunset Ln', 'Lakeside', '555-1010');

```

```

INSERT INTO eEmployees (employee_id, store_id, fname, lname, job,
salary, hiredate) VALUES (7001, 4001, 'John', 'Smith', 'Manager',
6500.00, DATE '2020-01-15');
INSERT INTO eEmployees (employee_id, store_id, fname, lname, job,
manager_id, salary, hiredate) VALUES (7002, 4002, 'Emily', 'Johnson',
'Cashier', 7001, 3200.00, DATE '2021-03-20');
INSERT INTO eEmployees (employee_id, store_id, fname, lname, job,
manager_id, salary, hiredate) VALUES (7003, 4003, 'Michael',
'Williams', 'Supervisor', 7001, 4800.00, DATE '2019-07-10');
INSERT INTO eEmployees (employee_id, store_id, fname, lname, job,
manager_id, salary, hiredate) VALUES (7004, 4004, 'Sarah', 'Brown',
'Sales Associate', 7003, 3500.00, DATE '2022-02-18');
INSERT INTO eEmployees (employee_id, store_id, fname, lname, job,
manager_id, salary, hiredate) VALUES (7005, 4005, 'David', 'Jones',
'Manager', 7001, 6700.00, DATE '2018-11-05');

```

```

INSERT INTO eEmployees (employee_id, store_id, fname, lname, job,
manager_id, salary, hiredate) VALUES (7006, 4006, 'Sophia', 'Garcia',
'Cashier', 7005, 3100.00, DATE '2023-01-09');
INSERT INTO eEmployees (employee_id, store_id, fname, lname, job,
manager_id, salary, hiredate) VALUES (7007, 4007, 'James', 'Miller',
'Supervisor', 7005, 5000.00, DATE '2020-09-14');
INSERT INTO eEmployees (employee_id, store_id, fname, lname, job,
manager_id, salary, hiredate) VALUES (7008, 4008, 'Olivia', 'Davis',
'Sales Associate', 7001, 3400.00, DATE '2021-12-01');
INSERT INTO eEmployees (employee_id, store_id, fname, lname, job,
manager_id, salary, hiredate) VALUES (7009, 4009, 'Daniel', 'Wilson',
'Manager', 7001, 6900.00, DATE '2017-06-25');
INSERT INTO eEmployees (employee_id, store_id, fname, lname, job,
manager_id, salary, hiredate) VALUES (7010, 4010, 'Emma', 'Taylor',
'Cashier', 7002, 3300.00, DATE '2022-08-30');

```

```

INSERT INTO ePromotion (promotion_id, minretail, maxretail, gift_id)
VALUES (1, 10.00, 20.00, 1);
INSERT INTO ePromotion (promotion_id, minretail, maxretail, gift_id)
VALUES (2, 20.00, 30.00, 2);
INSERT INTO ePromotion (promotion_id, minretail, maxretail, gift_id)
VALUES (3, 30.00, 40.00, 3);
INSERT INTO ePromotion (promotion_id, minretail, maxretail, gift_id)
VALUES (4, 40.00, 50.00, 4);
INSERT INTO ePromotion (promotion_id, minretail, maxretail, gift_id)
VALUES (5, 50.00, 70.00, 5);
INSERT INTO ePromotion (promotion_id, minretail, maxretail, gift_id)
VALUES (6, 70.00, 90.00, 6);
INSERT INTO ePromotion (promotion_id, minretail, maxretail, gift_id)
VALUES (7, 90.00, 120.00, 7);
INSERT INTO ePromotion (promotion_id, minretail, maxretail, gift_id)
VALUES (8, 120.00, 150.00, 8);
INSERT INTO ePromotion (promotion_id, minretail, maxretail, gift_id)
VALUES (9, 150.00, 200.00, 9);
INSERT INTO ePromotion (promotion_id, minretail, maxretail, gift_id)
VALUES (10, 200.00, 300.00, 10);

```

```
INSERT INTO eCategory (category_id, category_title, promotion_id)
VALUES (1, 'Computers', 1);
INSERT INTO eCategory (category_id, category_title, promotion_id)
VALUES (2, 'Accessories', 2);
INSERT INTO eCategory (category_id, category_title, promotion_id)
VALUES (3, 'Monitors', 3);
INSERT INTO eCategory (category_id, category_title, promotion_id)
VALUES (4, 'Printers', 4);
INSERT INTO eCategory (category_id, category_title, promotion_id)
VALUES (5, 'Furniture', 5);
INSERT INTO eCategory (category_id, category_title, promotion_id)
VALUES (6, 'Audio', 6);
INSERT INTO eCategory (category_id, category_title, promotion_id)
VALUES (7, 'Cameras', 7);
INSERT INTO eCategory (category_id, category_title, promotion_id)
VALUES (8, 'Storage', 8);
INSERT INTO eCategory (category_id, category_title, promotion_id)
VALUES (9, 'Office', 9);
INSERT INTO eCategory (category_id, category_title, promotion_id)
VALUES (10, 'Other', 10);
```

```
INSERT INTO eItems (item_id, item_name, category_id, cost, retail,
discount, vendor_id) VALUES (3001, 'Laptop', 1, 650.00, 899.99, 50.00,
101);
INSERT INTO eItems (item_id, item_name, category_id, cost, retail,
discount, vendor_id) VALUES (3002, 'Mouse', 2, 10.50, 19.99, 2.00,
102);
INSERT INTO eItems (item_id, item_name, category_id, cost, retail,
discount, vendor_id) VALUES (3003, 'Keyboard', 2, 20.00, 39.99, 5.00,
103);
INSERT INTO eItems (item_id, item_name, category_id, cost, retail,
discount, vendor_id) VALUES (3004, 'Monitor', 3, 120.00, 189.99,
15.00, 104);
```

```
INSERT INTO eItems (item_id, item_name, category_id, cost, retail,
discount, vendor_id) VALUES (3005, 'Printer', 4, 85.00, 149.99, 10.00,
105);
INSERT INTO eItems (item_id, item_name, category_id, cost, retail,
discount, vendor_id) VALUES (3006, 'Desk', 5, 75.00, 129.99, 0.00,
106);
INSERT INTO eItems (item_id, item_name, category_id, cost, retail,
discount, vendor_id) VALUES (3007, 'Chair', 5, 45.00, 89.99, 5.00,
107);
INSERT INTO eItems (item_id, item_name, category_id, cost, retail,
discount, vendor_id) VALUES (3008, 'Headset', 6, 25.00, 49.99, 3.00,
108);
INSERT INTO eItems (item_id, item_name, category_id, cost, retail,
discount, vendor_id) VALUES (3009, 'Webcam', 7, 35.00, 69.99, 7.00,
109);
INSERT INTO eItems (item_id, item_name, category_id, cost, retail,
discount, vendor_id) VALUES (3010, 'USB Drive', 8, 8.00, 14.99, 1.00,
110);
INSERT INTO eItems (item_id, item_name, category_id, cost, retail,
discount, vendor_id) VALUES (3011, 'Tablet', 1, 220.00, 349.99, 20.00,
101);
INSERT INTO eItems (item_id, item_name, category_id, cost, retail,
discount, vendor_id) VALUES (3012, 'Speaker', 6, 35.00, 69.99, 5.00,
108);
INSERT INTO eItems (item_id, item_name, category_id, cost, retail,
discount, vendor_id) VALUES (3013, 'Scanner', 4, 90.00, 159.99, 10.00,
105);
INSERT INTO eItems (item_id, item_name, category_id, cost, retail,
discount, vendor_id) VALUES (3014, 'Router', 2, 45.00, 79.99, 8.00,
102);
INSERT INTO eItems (item_id, item_name, category_id, cost, retail,
discount, vendor_id) VALUES (3015, 'Hard Drive', 8, 55.00, 99.99,
12.00, 110);
INSERT INTO eItems (item_id, item_name, category_id, cost, retail,
discount, vendor_id) VALUES (3016, 'Smartwatch', 7, 80.00, 149.99,
15.00, 109);
```

```

INSERT INTO eItems (item_id, item_name, category_id, cost, retail,
discount, vendor_id) VALUES (3017, 'Office Lamp', 9, 18.00, 39.99,
3.00, 106);
INSERT INTO eItems (item_id, item_name, category_id, cost, retail,
discount, vendor_id) VALUES (3018, 'Notebook', 9, 5.00, 12.99, 1.00,
107);
INSERT INTO eItems (item_id, item_name, category_id, cost, retail,
discount, vendor_id) VALUES (3019, 'Power Bank', 10, 20.00, 39.99,
4.00, 103);
INSERT INTO eItems (item_id, item_name, category_id, cost, retail,
discount, vendor_id) VALUES (3020, 'HDMI Cable', 2, 6.00, 14.99, 2.00,
104);

```

```

INSERT INTO eStocks (store_id, item_id, stock_quantity) VALUES (4001,
3001, 25);
INSERT INTO eStocks (store_id, item_id, stock_quantity) VALUES (4002,
3002, 50);
INSERT INTO eStocks (store_id, item_id, stock_quantity) VALUES (4003,
3003, 35);
INSERT INTO eStocks (store_id, item_id, stock_quantity) VALUES (4004,
3004, 20);
INSERT INTO eStocks (store_id, item_id, stock_quantity) VALUES (4005,
3005, 15);
INSERT INTO eStocks (store_id, item_id, stock_quantity) VALUES (4006,
3006, 30);
INSERT INTO eStocks (store_id, item_id, stock_quantity) VALUES (4007,
3007, 40);
INSERT INTO eStocks (store_id, item_id, stock_quantity) VALUES (4008,
3008, 45);
INSERT INTO eStocks (store_id, item_id, stock_quantity) VALUES (4009,
3009, 18);
INSERT INTO eStocks (store_id, item_id, stock_quantity) VALUES (4010,
3010, 60);

```

```

INSERT INTO eReturnstocks (return_id, item_id, vendor_id, store_id,
returndate) VALUES (6001, 3001, 101, 4001, DATE '2023-11-01');

```

```
INSERT INTO eReturnstocks (return_id, item_id, vendor_id, store_id, returndate) VALUES (6002, 3002, 102, 4002, DATE '2023-11-03');
INSERT INTO eReturnstocks (return_id, item_id, vendor_id, store_id, returndate) VALUES (6003, 3003, 103, 4003, DATE '2023-11-05');
INSERT INTO eReturnstocks (return_id, item_id, vendor_id, store_id, returndate) VALUES (6004, 3004, 104, 4004, DATE '2023-11-07');
INSERT INTO eReturnstocks (return_id, item_id, vendor_id, store_id, returndate) VALUES (6005, 3005, 105, 4005, DATE '2023-11-09');
INSERT INTO eReturnstocks (return_id, item_id, vendor_id, store_id, returndate) VALUES (6006, 3006, 106, 4006, DATE '2023-11-11');
INSERT INTO eReturnstocks (return_id, item_id, vendor_id, store_id, returndate) VALUES (6007, 3007, 107, 4007, DATE '2023-11-13');
INSERT INTO eReturnstocks (return_id, item_id, vendor_id, store_id, returndate) VALUES (6008, 3008, 108, 4008, DATE '2023-11-15');
INSERT INTO eReturnstocks (return_id, item_id, vendor_id, store_id, returndate) VALUES (6009, 3009, 109, 4009, DATE '2023-11-17');
INSERT INTO eReturnstocks (return_id, item_id, vendor_id, store_id, returndate) VALUES (6010, 3010, 110, 4010, DATE '2023-11-19');
```

```
INSERT INTO eOrderItems (order_id, item_id, item_quantity, payeach) VALUES (2001, 3001, 2, 19.99);
INSERT INTO eOrderItems (order_id, item_id, item_quantity, payeach) VALUES (2003, 3002, 1, 49.99);
INSERT INTO eOrderItems (order_id, item_id, item_quantity, payeach) VALUES (2003, 3003, 3, 15.50);
INSERT INTO eOrderItems (order_id, item_id, item_quantity, payeach) VALUES (2004, 3004, 2, 29.95);
INSERT INTO eOrderItems (order_id, item_id, item_quantity, payeach) VALUES (2005, 3005, 5, 9.99);
INSERT INTO eOrderItems (order_id, item_id, item_quantity, payeach) VALUES (2007, 3006, 1, 89.50);
INSERT INTO eOrderItems (order_id, item_id, item_quantity, payeach) VALUES (2007, 3007, 4, 12.75);
INSERT INTO eOrderItems (order_id, item_id, item_quantity, payeach) VALUES (2007, 3008, 2, 39.99);
```

```
INSERT INTO eOrderItems (order_id, item_id, item_quantity, payeach)
VALUES (2002, 3009, 1, 59.95);
INSERT INTO eOrderItems (order_id, item_id, item_quantity, payeach)
VALUES (2005, 3010, 3, 24.99);
INSERT INTO eOrderItems (order_id, item_id, item_quantity, payeach)
VALUES (2006, 3009, 1, 59.95);
INSERT INTO eOrderItems (order_id, item_id, item_quantity, payeach)
VALUES (2008, 3001, 2, 19.99);
INSERT INTO eOrderItems (order_id, item_id, item_quantity, payeach)
VALUES (2009, 3003, 3, 15.50);
INSERT INTO eOrderItems (order_id, item_id, item_quantity, payeach)
VALUES (2010, 3007, 4, 12.75);
```

```
INSERT INTO eWishlist (customer_id, item_id, added_date) VALUES (1001,
3001, DATE '2023-02-01');
INSERT INTO eWishlist (customer_id, item_id, added_date) VALUES (1001,
3004, DATE '2023-02-15');
INSERT INTO eWishlist (customer_id, item_id, added_date) VALUES (1002,
3011, DATE '2023-03-01');
INSERT INTO eWishlist (customer_id, item_id, added_date) VALUES (1003,
3006, DATE '2023-03-10');
INSERT INTO eWishlist (customer_id, item_id, added_date) VALUES (1004,
3016, DATE '2023-04-05');
INSERT INTO eWishlist (customer_id, item_id, added_date) VALUES (1005,
3008, DATE '2023-05-01');
INSERT INTO eWishlist (customer_id, item_id, added_date) VALUES (1006,
3015, DATE '2023-06-01');
INSERT INTO eWishlist (customer_id, item_id, added_date) VALUES (1007,
3002, DATE '2023-06-15');
INSERT INTO eWishlist (customer_id, item_id, added_date) VALUES (1008,
3019, DATE '2023-07-20');
INSERT INTO eWishlist (customer_id, item_id, added_date) VALUES (1010,
3005, DATE '2023-09-10');
```

DM_eCommerce.sql

SQL Worksheet: History

connect_oracle23a

Worksheet Query Builder

```

select * from eCustomers;
select * from eOrders;
select * from eVendors;
select * from eGift;
select * from eStores;
select * from eEmployees;
select * from ePromotion;
select * from eCategory;
select * from eItems;
select * from eStocks;
select * from eReturnstocks;
select * from eOrderItems;

```

Script Output Query Result

All Rows Fetched: 10 in 0.012 seconds

	CUSTOMER_ID	LASTNAME	FIRSTNAME	CUSTOMER_ADDRESS	EMAIL	REGISTERDATE
1	1001	Smith	John	123 Maple St	john@email.com	23-01-15
2	1002	Johnson	Emily	456 Oak Ave	emily@email.com	23-02-10
3	1003	Williams	Michael	789 Pine Rd	michael@email.com	23-03-05
4	1004	Brown	Sarah	321 Cedar Dr	sarah@email.com	23-03-22
5	1005	Jones	David	654 Birch Ln	david@email.com	23-04-18
6	1006	Garcia	Sophia	987 Willow St	sophia@email.com	23-05-11
7	1007	Miller	James	741 Elm St	james@email.com	23-06-03
8	1008	Davis	Olivia	852 Cherry Ave	olivia@email.com	23-07-09
9	1009	Wilson	Daniel	963 Aspen Ct	daniel@email.com	23-08-14
10	1010	Taylor	Emma	159 Spruce Blvd	emma@email.com	23-09-01

DM_eCommerce.sql

SQL Worksheet: History

connect_oracle23a

Worksheet Query Builder

```

select * from eCustomers;
select * from eOrders;
select * from eVendors;
select * from eGift;
select * from eStores;
select * from eEmployees;
select * from ePromotion;
select * from eCategory;
select * from eItems;
select * from eStocks;
select * from eReturnstocks;
select * from eOrderItems;

```

Script Output Query Result

All Rows Fetched: 10 in 0.01 seconds

	ORDER_ID	CUSTOMER_ID	ORDERDATE	SHIPDATE	SHIPCOST	SHIPADDRESS
1	2001	1001	23-10-01	23-10-03	8.99	123 Maple St
2	2002	1002	23-10-02	23-10-05	12.5	456 Oak Ave
3	2003	1003	23-10-04	23-10-06	6.75	789 Pine Rd
4	2004	1004	23-10-05	23-10-08	9.99	321 Cedar Dr
5	2005	1005	23-10-06	23-10-07	5.5	654 Birch Ln
6	2006	1006	23-10-07	23-10-10	11.25	987 Willow St
7	2007	1007	23-10-08	23-10-09	7.95	741 Elm St
8	2008	1008	23-10-09	23-10-12	13.4	852 Cherry Ave
9	2009	1009	23-10-10	23-10-13	10	963 Aspen Ct
10	2010	1010	23-10-11	23-10-14	8.25	159 Spruce Blvd

DM_eCommerce.sql

SQL Worksheet: History

connect_oracle23a

Worksheet Query Builder

```

select * from eCustomers;
select * from eOrders;
select * from eVendors;
select * from eGift;
select * from eStores;
select * from eEmployees;
select * from ePromotion;
select * from eCategory;
select * from eItems;
select * from eStocks;
select * from eReturnstocks;
select * from eOrderItems;

```

Script Output Query Result

All Rows Fetched: 10 in 0.011 seconds

	VENDOR_ID	VENDOR_NAME	CONTACTNUMBER	CONTACTNAME	VENDOR_ADDRESS	CONTACTDATE	BUILDDATE
1	101	TechSupply	555-1001	Alice Brown	12 Market St	23-06-15	18-04-10
2	102	OfficePlus	555-1002	Brian Smith	45 King Ave	23-07-10	19-02-18
3	103	GadgetWorld	555-1003	Carol Davis	78 Queen Rd	23-08-05	20-05-12
4	104	HomeGoods	555-1004	David Wilson	91 Elm St	23-06-20	17-09-30
5	105	FurnitureCo	555-1005	Emma Taylor	23 Oak Dr	23-09-12	16-11-22
6	106	PrintMaster	555-1006	Frank Miller	56 Pine Blvd	23-07-18	19-08-15
7	107	DigitalHub	555-1007	Grace Johnson	89 Cedar Ave	23-10-01	21-01-05
8	108	AccessoryPro	555-1008	Henry Moore	14 Birch Ln	23-08-28	20-06-10
9	109	SmartDevices	555-1009	Isabella White	37 Willow Ct	23-09-30	18-12-08
10	110	ComputerZone	555-1010	Jack Harris	62 Maple Rd	23-10-15	17-03-25

The image displays three sequential screenshots of the SQL Developer interface, each showing a different SQL query and its corresponding result set.

Top Screenshot: The SQL query pane contains a query that selects all columns from ten tables: eCustomers, eOrders, eVendors, eGift, eStores, eEmployees, ePromotion, eCategory, eItems, eStocks, eReturnstocks, and eOrderItems. The 'Query Result' pane shows 10 rows of data with columns GIFT_ID, GIFT_NAME, and GIFT_QUANTITY.

GIFT_ID	GIFT_NAME	GIFT_QUANTITY
1	1 Free Mouse	50
2	2 USB Cable	100
3	3 Notebook	75
4	4 Pen Set	120
5	5 Phone Stand	60
6	6 Screen Cleaner	80
7	7 Laptop Bag	40
8	8 Keyboard Cover	90
9	9 Coffee Mug	55
10	10 Sticker Pack	150

Middle Screenshot: The SQL query pane contains a query that selects all columns from the eStores table. The 'Query Result' pane shows 10 rows of data with columns STORE_ID, STORE_ADDRESS, STORE_TITLE, and STORE_CONTACT.

STORE_ID	STORE_ADDRESS	STORE_TITLE	STORE_CONTACT
1	4001 123 Main St	Downtown	555-1001
2	4002 45 King Rd	Westside	555-1002
3	4003 89 Queen Ave	Eastside	555-1003
4	4004 12 River St	North Mall	555-1004
5	4005 77 Lake Dr	South Plaza	555-1005
6	4006 91 Hill Rd	Central	555-1006
7	4007 63 Park Ave	Riverside	555-1007
8	4008 28 Green St	City Mall	555-1008
9	4009 14 Oak Blvd	Uptown	555-1009
10	4010 55 Sunset Ln	Lakeside	555-1010

Bottom Screenshot: The SQL query pane contains a query that selects all columns from the eEmployees table. The 'Query Result' pane shows 10 rows of data with columns EMPLOYEE_ID, STORE_ID, FNAME, LNAME, JOB, MANAGER_ID, SALARY, and HIREDATE.

EMPLOYEE_ID	STORE_ID	FNAME	LNAME	JOB	MANAGER_ID	SALARY	HIREDATE
1	7001	John	Smith	Manager	(null)	6500	20-01-15
2	7002	Emily	Johnson	Cashier	7001	3200	21-03-20
3	7003	Michael	Williams	Supervisor	7001	4800	19-07-10
4	7004	Sarah	Brown	Sales Associate	7003	3500	22-02-18
5	7005	David	Jones	Manager	7001	6700	18-11-05
6	7006	Sophia	Garcia	Cashier	7005	3100	23-01-09
7	7007	James	Miller	Supervisor	7005	5000	20-09-14
8	7008	Olivia	Davis	Sales Associate	7001	3400	21-12-01
9	7009	Daniel	Wilson	Manager	7001	6900	17-06-25
10	7010	Emma	Taylor	Cashier	7002	3300	22-08-30

DM4_eCommerce.sql

SQL Worksheet: History

connect_oracle23ai

Worksheet Query Builder

```

select * from eCustomers;
select * from eOrders;
select * from eVendors;
select * from eGift;
select * from eStores;
select * from eEmployees;
select * from ePromotion;
select * from eCategory;
select * from eItems;
select * from eStocks;
select * from eReturnstocks;
select * from eOrderItems;

```

Script Output Query Result

All Rows Fetched: 10 in 0.009 seconds

	PROMOTION_ID	MINRETAIL	MAXRETAIL	GIFT_ID
1	1	10	20	1
2	2	20	30	2
3	3	30	40	3
4	4	40	50	4
5	5	50	70	5
6	6	70	90	6
7	7	90	120	7
8	8	120	150	8
9	9	150	200	9
10	10	200	300	10

DM4_eCommerce.sql

SQL Worksheet: History

connect_oracle23ai

Worksheet Query Builder

```

select * from eCustomers;
select * from eOrders;
select * from eVendors;
select * from eGift;
select * from eStores;
select * from eEmployees;
select * from ePromotion;
select * from eCategory;
select * from eItems;
select * from eStocks;
select * from eReturnstocks;
select * from eOrderItems;

```

Script Output Query Result

All Rows Fetched: 10 in 0.009 seconds

	CATEGORY_ID	CATEGORY_TITLE	PROMOTION_ID
1	1	Computers	1
2	2	Accessories	2
3	3	Monitors	3
4	4	Printers	4
5	5	Furniture	5
6	6	Audio	6
7	7	Cameras	7
8	8	Storage	8
9	9	Office	9
10	10	Other	10

DM4_eCommerce.sql

SQL Worksheet: History

connect_oracle23ai

Worksheet Query Builder

```

select * from eCustomers;
select * from eOrders;
select * from eVendors;
select * from eGift;
select * from eStores;
select * from eEmployees;
select * from ePromotion;
select * from eCategory;
select * from eItems;
select * from eStocks;
select * from eReturnstocks;
select * from eOrderItems;

```

Script Output Query Result

All Rows Fetched: 20 in 0.011 seconds

	ITEM_ID	ITEM_NAME	CATEGORY_ID	COST	RETAIL	DISCOUNT	VENDOR_ID
1	3001	Laptop	1	650	899.99	50	101
2	3002	Mouse	2	10.5	19.99	2	102
3	3003	Keyboard	2	20	39.99	5	103
4	3004	Monitor	3	120	189.99	15	104
5	3005	Printer	4	85	149.99	10	105
6	3006	Desk	5	75	129.99	0	106
7	3007	Chair	5	45	89.99	5	107
8	3008	Headset	6	25	49.99	3	108
9	3009	Webcam	7	35	69.99	7	109
10	3010	USB Drive	8	8	14.99	1	110
11	3011	Tablet	1	220	349.99	20	101
12	3012	Headphones	2	35	69.99	5	108

The image displays three sequential screenshots of the SQL Developer interface, showing different queries and their results in the DM4_eCommerce.sql workspace. The interface includes a toolbar with icons for connections, worksheets, and query building, and a status bar indicating the number of rows fetched and execution time.

First Screenshot: The query selects data from various tables, and the result set shows 10 rows of stock quantities.

	STORE_ID	ITEM_ID	STOCK_QUANTITY
1	4001	3001	25
2	4002	3002	50
3	4003	3003	35
4	4004	3004	20
5	4005	3005	15
6	4006	3006	30
7	4007	3007	40
8	4008	3008	45
9	4009	3009	18
10	4010	3010	60

Second Screenshot: The query selects data from various tables, and the result set shows 10 rows of return information.

	RETURN_ID	ITEM_ID	VENDOR_ID	STORE_ID	RETURNDATE
1	6001	3001	101	4001	23-11-01
2	6002	3002	102	4002	23-11-03
3	6003	3003	103	4003	23-11-05
4	6004	3004	104	4004	23-11-07
5	6005	3005	105	4005	23-11-09
6	6006	3006	106	4006	23-11-11
7	6007	3007	107	4007	23-11-13
8	6008	3008	108	4008	23-11-15
9	6009	3009	109	4009	23-11-17
10	6010	3010	110	4010	23-11-19

Third Screenshot: The query selects data from various tables, and the result set shows 10 rows of order and item information.

	ORDER_ID	ITEM_ID	ITEM_QUANTITY	PAYEACH
1	2001	3001	2	19.99
2	2002	3002	1	49.99
3	2003	3003	3	15.5
4	2004	3004	2	29.95
5	2005	3005	5	9.99
6	2006	3006	1	89.5
7	2007	3007	4	12.75
8	2008	3008	2	39.99
9	2009	3009	1	59.95
10	2010	3010	3	24.99

The screenshot shows the SQL Developer interface with a query window titled 'DME_eCommerce.sql'. The query is a multi-table SELECT statement. Below the query, the 'Query Result' window displays 10 rows of data with columns: CUSTOMER_ID, ITEM_ID, and ADDED_DATE.

```
select * from eCustomers;
select * from eOrders;
select * from eVendors;
select * from eGift;
select * from eStores;
select * from eEmployees;
select * from ePromotion;
select * from eCategory;
select * from eItems;
select * from eStocks;
select * from eReturnstocks;
select * from eOrderItems;
select * from ewishlist;
```

	CUSTOMER_ID	ITEM_ID	ADDED_DATE
1	1001	3001	23-02-01
2	1001	3004	23-02-15
3	1002	3011	23-03-01
4	1003	3006	23-03-10
5	1004	3016	23-04-05
6	1005	3008	23-05-01
7	1006	3015	23-06-01
8	1007	3002	23-06-15
9	1008	3019	23-07-20
10	1010	3005	23-09-10

Phase 5 :

Business Requirements:

1. Product Discovery: Search products based on category.

```
SELECT i.item_name, e.category_title
FROM eitems i JOIN ecategory e USING (category_id)
WHERE e.category_title='Computers';
```

The screenshot shows a database query interface with a 'Worksheet' tab and a 'Query Builder' tab. The SQL query is displayed in a text area:

```
SELECT i.item_name, e.category_title
FROM eitems i JOIN ecategory e USING (category_id)
WHERE e.category_title='Computers';
```

Below the query, the 'Query Result' tab is active, showing the results of the query. The results are displayed in a table with two columns: 'ITEM_NAME' and 'CATEGORY_TITLE'. The table contains two rows:

	ITEM_NAME	CATEGORY_TITLE
1	Laptop	Computers
2	Tablet	Computers

The status bar indicates 'All Rows Fetched: 2 in 0.018 seconds'.

```
SELECT i.item_name, e.category_title
FROM eitems i JOIN ecategory e USING (category_id)
WHERE category_id=10;
```

The screenshot shows a database query interface with a 'Worksheet' tab and a 'Query Builder' tab. The SQL query is displayed in a text area:

```
SELECT i.item_name, e.category_title
FROM eitems i JOIN ecategory e USING (category_id)
WHERE category_id=10;
```

Below the query, the 'Query Result' tab is active, showing the results of the query. The results are displayed in a table with two columns: 'ITEM_NAME' and 'CATEGORY_TITLE'. The table contains one row:

	ITEM_NAME	CATEGORY_TITLE
1	Power Bank	Other

The status bar indicates 'All Rows Fetched: 1 in 0.018 seconds'.

```
SELECT i.item_name, e.category_title
FROM eitems i JOIN ecategory e ON i.category_id=e.category_id
WHERE i.category_id=5;
```

Worksheet Query Builder

```
SELECT i.item_name, e.category_title
FROM eitems i JOIN ecategory e ON i.category_id=e.category_id
WHERE i.category_id=5;
```

Query Result x

SQL | All Rows Fetched: 2 in 0.027 seconds

	ITEM_NAME	CATEGORY_TITLE
1	Desk	Furniture
2	Chair	Furniture

2. Shopping Cart Operations: Manage shopping cart.

```
SELECT i.item_name, oi.item_quantity, oi.payeach
FROM eitems i JOIN eorderitems oi USING (item_id)
WHERE oi.order_id='2003';
```

Worksheet Query Builder

```
SELECT i.item_name, oi.item_quantity, oi.payeach
FROM eitems i JOIN eorderitems oi USING (item_id)
WHERE oi.order_id='2003';
```

Query Result x

SQL | All Rows Fetched: 2 in 0.027 seconds

	ITEM_NAME	ITEM_QUANTITY	PAYEACH
1	Mouse	1	49.99
2	Keyboard	3	15.5

```
SELECT i.item_name, oi.item_quantity, oi.payeach
FROM eitems i JOIN eorderitems oi ON i.item_id=oi.item_id
WHERE oi.order_id='2007';
```

Worksheet Query Builder

```
SELECT i.item_name, oi.item_quantity, oi.payeach
FROM eitems i JOIN eorderitems oi ON i.item_id=oi.item_id
WHERE oi.order_id='2007';
```

Query Result x

SQL | All Rows Fetched: 3 in 0.022 seconds

	ITEM_NAME	ITEM_QUANTITY	PAYEACH
1	Desk	1	89.5
2	Chair	4	12.75
3	Headset	2	39.99

3. User Account Management: Register/unregister user.

```
SELECT INITCAP(firstname) || ' ' || INITCAP(lastname) AS "Customer
Name", customer_address, email
FROM ecustomers
WHERE registerdate='2023-05-11';
```

Worksheet		Query Builder							
		<pre>//Register name SELECT INITCAP(firstname) ' ' INITCAP(lastname) AS "Customer Name", customer_address, email FROM ecustomers WHERE registerdate='2023-05-11';</pre>							
Query Result x SQL All Rows Fetched: 1 in 0.02 seconds <table><thead><tr><th>Customer Name</th><th>CUSTOMER_ADDRESS</th><th>EMAIL</th></tr></thead><tbody><tr><td>1 Sophia Garcia</td><td>987 Willow St</td><td>sophia@email.com</td></tr></tbody></table>				Customer Name	CUSTOMER_ADDRESS	EMAIL	1 Sophia Garcia	987 Willow St	sophia@email.com
Customer Name	CUSTOMER_ADDRESS	EMAIL							
1 Sophia Garcia	987 Willow St	sophia@email.com							

```
SELECT INITCAP(firstname) || ' ' || INITCAP(lastname) AS "Customer
Name", customer_address, email
FROM ecustomers
WHERE registerdate='1111-11-11';
```

Worksheet		Query Builder	
		<pre>// Unregister name (registerdate='1111-11-11' is the sign of unregistering) SELECT INITCAP(firstname) ' ' INITCAP(lastname) AS "Customer Name", customer_address, email FROM ecustomers WHERE registerdate='1111-11-11';</pre>	

Query Result x			
    SQL All Rows Fetched: 1 in 0.019 seconds			
Customer Name	CUSTOMER_ADDR...	EMAIL	
1 Emma Taylor	159 Spruce Blvd	emma@email.com	

4. Logistics Tracking: Track shipping status.

```
SELECT order_id, shipdate, orderdate, (shipdate-orderdate) AS "Delay"
FROM eorders
WHERE order_id='2008';
```

Worksheet

Query Builder

```
SELECT order_id, shipdate, orderdate, (shipdate-orderdate) AS "Delay"
FROM eorders
WHERE order_id='2008';
```

▶ Query Result x



SQL | All Rows Fetched: 1 in 0.016 seconds

	ORDER_ID	SHIPDATE	ORDERDATE	Delay
1	2008	23-10-12	23-10-09	3

5. Supply Chain Management: Manage vendors.

```
SELECT i.item_name, v.vendor_name, v.vendor_address, r.returndate
FROM eitems i JOIN evendors v USING (vendor_id)
        JOIN ereturnstocks r USING (vendor_id)
WHERE vendor_id=101;
```

Worksheet

Query Builder

3. Supply Chain Management: Manage vendors:

```
SELECT v.vendor_name, v.contractdate, i.item_name
FROM evendors v JOIN eitems i ON v.vendor_id=i.vendor_id
WHERE contractdate='2023-10-15';
```

Worksheet

Query Builder

```
SELECT v.vendor_name, v.contractdate, i.item_name
FROM evendors v JOIN eitems i ON v.vendor_id=i.vendor_id
WHERE contractdate='2023-10-15';
```

▶ Query Result x

| All Rows Fetched: 2 in 0.029 seconds

	VENDOR_NAME	CONTRACTDATE	ITEM_NAME
1	ComputerZone	23-10-15	USB Drive
2	ComputerZone	23-10-15	Hard Drive

6. Executive Reporting: Generate monthly report (e.g., sales per month).

```
select
    to_char(orderdate, 'YYYY-MM') as "Sales Month",
    sum(item_quantity) as "Total Units Sold",
    sum(item_quantity * payeach) as "Total Revenue"
from eorders join eorderitems using(order_id)
group by to_char(orderdate, 'YYYY-MM')
order by "Sales Month";
```

Worksheet

Query Builder

```
--select * from eorders;
--6. Executive Reporting: Generate monthly report (e.g., sales per month).
select
    to_char(orderdate, 'YYYY-MM') as "Sales Month",
    sum(item_quantity) as "Total Units Sold",
    sum(item_quantity * payeach) as "Total Revenue"
from eorders join eorderitems using(order_id)
group by to_char(orderdate, 'YYYY-MM')
order by "Sales Month";
```

Query Result x

Query Result 1 x

SQL | All Rows Fetched: 1 in 0.003 seconds

	Sales Month	Total Units Sold	Total Revenue
1	2023-10	24	601.72

7. Inventory Analytics: Identify the most popular product(s).

```
select item_id, item_name,
    sum(item_quantity) as "Total Units Sold",
    sum(item_quantity * payeach) as "Total Revenue"
from eitems join eorderitems using(item_id)
```

```
group by item_id, item_name
order by "Total Units Sold" desc;
```

Worksheet

Query Builder

```
--7. Inventory Analytics: Identify the most popular product(s).
```

```
select item_id, item_name,  
       sum(item_quantity) as "Total Units Sold",  
       sum(item_quantity * payeach) as "Total Revenue"  
from eitems join eorderitems using(item_id)  
group by item_id, item_name  
order by "Total Units Sold" desc;
```

Query Result x

SQL | All Rows Fetched: 10 in 0.007 seconds

	ITEM_ID	ITEM_NAME	Total Units Sold	Total Revenue
1	3005	Printer	5	49.95
2	3007	Chair	4	51
3	3003	Keyboard	3	46.5
4	3010	USB Drive	3	74.97
5	3008	Headset	2	79.98
6	3004	Monitor	2	59.9
7	3001	Laptop	2	39.98
8	3009	Webcam	1	59.95
9	3002	Mouse	1	49.99
10	3006	Desk	1	89.5

8. Customer Segmentation: Categorize customers based on his/her purchase history.

```
select
  c.customer_id,
  c.firstname || ' ' || c.lastname,
  nvl(sum(oi.item_quantity * oi.payeach), 0) as "Total Spend",
  case
    when sum(oi.item_quantity * oi.payeach) >= 80 then 'VIP'
    when sum(oi.item_quantity * oi.payeach) between 45 and 79.99
then 'Gold'
    when sum(oi.item_quantity * oi.payeach) between 1 and 44.99
then 'Silver'
    else 'Inactive'
  end as "Customer Tier"
from ecustomers c
left join eorders o on c.customer_id = o.customer_id
left join eorderitems oi on o.order_id = oi.order_id
```

```
group by c.customer_id, c.firstname || ' ' || c.lastname
order by "Total Spend" desc;
```

Worksheet Query Builder

--8. Customer Segmentation: Categorize customers based on his/her purchase history.

```
select
  c.customer_id,
  c.firstname || ' ' || c.lastname,
  nvl(sum(oi.item_quantity * oi.payeach), 0) as "Total Spend",
  case
    when sum(oi.item_quantity * oi.payeach) >= 80 then 'VIP'
    when sum(oi.item_quantity * oi.payeach) between 45 and 79.99 then 'Gold'
    when sum(oi.item_quantity * oi.payeach) between 1 and 44.99 then 'Silver'
    else 'Inactive'
  end as "Customer Tier"
from ecustomers c
left join eorders o on c.customer_id = o.customer_id
left join eorderitems oi on o.order_id = oi.order_id
group by c.customer_id, c.firstname || ' ' || c.lastname
order by "Total Spend" desc;
```

Query Result x

All Rows Fetched: 10 in 0.008 seconds

	CUSTOMER_ID	C.FIRSTNAME " " C.LASTNAME	Total Spend	Customer Tier
1	1006	Sophia Garcia	89.5	VIP
2	1008	Olivia Davis	79.98	Gold
3	1010	Emma Taylor	74.97	Gold
4	1009	Daniel Wilson	59.95	Gold
5	1004	Sarah Brown	59.9	Gold
6	1007	James Miller	51	Gold
7	1002	Emily Johnson	49.99	Gold
8	1005	David Jones	49.95	Gold
9	1003	Michael Williams	46.5	Gold
10	1001	John Smith	39.98	Silver

9. Wish List Engagement: List all items on user's wish list.

```
select c.customer_id, c.firstname || ' ' || c.lastname as "Customer
Name", i.item_id, i.item_name, w.added_date
from ewishlist w
join ecustomers c on c.customer_id=w.customer_id
join eitems i on w.item_id=i.item_id
where c.customer_id=1001;
```

Worksheet

Query Builder

--9. Wish List Engagement: List all items on user's wish list.

```
select c.customer_id, c.firstname || ' ' || c.lastname as "Customer Name", i.item_id, i.item_name, w.added_date
from ewishlist w
join ecustomers c on c.customer_id=w.customer_id
join eitems i on w.item_id=i.item_id
where c.customer_id=1001;
```

Query Result x

Query Result 1 x

Query Result 2 x

 All Rows Fetched: 2 in 0.01 seconds

	CUSTOMER_ID	Customer Name	ITEM_ID	ITEM_NAME	ADDED_DATE
1	1001	John Smith	3001	Laptop	23-02-01
2	1001	John Smith	3004	Monitor	23-02-15

10. Customer Service Processing: Handle returns and refunds.

```
select rs.returndate, s.store_title, v.vendor_name, i.item_name, i.cost
as "Unit Cost"

from ereturnstocks rs

join estores s on rs.store_id = s.store_id

join EVENDORS v on rs.vendor_id = v.vendor_id

join EITEMS i on rs.item_id = i.item_id

order by rs.returndate desc;
```

Worksheet

Query Builder

--10. Customer Service Processing: Handle returns and refunds.
select rs.returndate, s.store_title, v.vendor_name, i.item_name, i.cost as "Unit Cost"
from ereturnstocks rs
join estores s on rs.store_id = s.store_id
join EVENDORS v on rs.vendor_id = v.vendor_id
join EITEMS i on rs.item_id = i.item_id
order by rs.returndate desc;

Query Result x

    SQL | All Rows Fetched: 10 in 0.003 seconds

	RETURNDATE	STORE_TITLE	VENDOR_NAME	ITEM_NAME	Unit Cost
1	23-11-19	Lakeside	ComputerZone	USB Drive	8
2	23-11-17	Uptown	SmartDevices	Webcam	35
3	23-11-15	City Mall	AccessoryPro	Headset	25
4	23-11-13	Riverside	DigitalHub	Chair	45
5	23-11-11	Central	PrintMaster	Desk	75
6	23-11-09	South Plaza	FurnitureCo	Printer	85
7	23-11-07	North Mall	HomeGoods	Monitor	120
8	23-11-05	Eastside	GadgetWorld	Keyboard	20
9	23-11-03	Westside	OfficePlus	Mouse	10.5
10	23-11-01	Downtown	TechSupply	Laptop	650